



نظریه‌ی زبان‌ها و ماشین‌ها

نیم‌سال دوم ۱۴۰۴-۰۵

استاد: دکتر محمد ایزدی

تمرین پنجم

پاسخ‌دهنده: معین آعلی - ۴۰۱۱۰۵۵۶۱

فهرست مسائل

۱		مسئله ۱
۲		مسئله ۲
۲		آ
۲		ب
۳		ج
۳		د
۴		مسئله ۳
۴		آ
۵		ب
۶		مسئله ۴
۷		مسئله ۵
۸		مسئله ۶

پاسخ مسئله‌ی ۱.

برای این‌که نشان دهیم زبان A یک زبان NP-complete است، باید دو شرط زیر را اثبات کنیم:

$$A \in NP \quad ۱.$$

۲. برای هر زبان $L \in NP$ ، کاهش زمان چندجمله‌ای $L \leq_P A$ وجود دارد.

طبق فرض مسئله $A \in P$ است. می‌دانیم که همیشه $P \subseteq NP$ برقرار است. بنابراین به طور مستقیم داریم $A \in NP$. فرض کنید L یک زبان دلخواه در NP باشد. طبق فرض $P = NP$ ، نتیجه می‌شود که $L \in P$ است. بنابراین، یک ماشین تورینگ قطعی مانند M_L وجود دارد که زبان L را در زمان چندجمله‌ای تصمیم‌گیری می‌کند.

از طرفی چون $A \neq \emptyset$ و $A \neq \Sigma^*$ است، حداقل یک رشته $w_{in} \in A$ و حداقل یک رشته $w_{out} \notin A$ وجود دارند.

حال تابع کاهش $f: \Sigma^* \rightarrow \Sigma^*$ را برای ورودی x به صورت زیر تعریف می‌کنیم:

$$f(x) = \begin{cases} w_{in} & x \in L \\ w_{out} & x \notin L \end{cases}$$

بر اساس تعریف تابع f داریم:

$$x \in L \iff f(x) = w_{in} \in A$$

$$x \notin L \iff f(x) = w_{out} \notin A$$

بنابراین $x \in L \iff f(x) \in A$ همواره برقرار است.

برای محاسبه $f(x)$ ، ابتدا ماشین M_L را روی x اجرا می‌کنیم که این کار طبق فرض $L \in P$ در زمان چندجمله‌ای انجام می‌شود. سپس بسته به نتیجه، یکی از دو رشته ثابت w_{in} یا w_{out} را در زمان $O(1)$ خروجی می‌دهیم. بنابراین تابع f در زمان چندجمله‌ای قابل محاسبه است.

نتیجه می‌گیریم که $L \leq_P A$ است و از آنجا که L یک زبان دلخواه در NP بود، شرط دوم نیز برقرار است.

پاسخ مسئله‌ی ۲.

فرض کنید L_1 و L_2 دو زبان دلخواه در کلاس مربوطه باشند.

آ

کلاس $\mathcal{P}$: چون $L_1, L_2 \in \mathcal{P}$ ، ماشین‌های تورینگ قطعی مانند M_1 و M_2 وجود دارند که به ترتیب L_1 و L_2 را در زمان چندجمله‌ای $O(p_1(n))$ و $O(p_2(n))$ تصمیم‌گیری می‌کنند. ماشین قطعی M را برای $L_1 \cup L_2$ روی ورودی x با طول n به صورت زیر می‌سازیم:

۱. ماشین $M_1(x)$ را اجرا کن. اگر پذیرفت، بپذیر.

۲. ماشین $M_2(x)$ را اجرا کن. اگر پذیرفت، بپذیر؛ در غیر این صورت رد کن.

زمان اجرای کل برابر $O(p_1(n) + p_2(n))$ است که چندجمله‌ای می‌باشد. بنابراین $L_1 \cup L_2 \in \mathcal{P}$.

کلاس $\mathcal{NP}$: چون $L_1, L_2 \in \mathcal{NP}$ ، ماشین‌های تورینگ غیرقطعی مانند N_1 و N_2 وجود دارند که در زمان چندجمله‌ای کار می‌کنند. ماشین غیرقطعی N را روی ورودی x به صورت زیر تعریف می‌کنیم:

۱. به صورت غیرقطعی یکی از دو مسیر زیر را انتخاب کن:

• $N_1(x)$ را اجرا کن.

• $N_2(x)$ را اجرا کن.

۲. اگر ماشین انتخاب‌شده ورودی را پذیرفت، بپذیر.

زمان اجرا حداکثر $\max(\text{poly}_1(n), \text{poly}_2(n))$ است، پس $L_1 \cup L_2 \in \mathcal{NP}$.

ب

کلاس $\mathcal{P}$: ماشین قطعی M برای $L_1 \cap L_2$ روی ورودی x :

۱. ماشین $M_1(x)$ را اجرا کن. اگر رد کرد، رد کن.

۲. ماشین $M_2(x)$ را اجرا کن. اگر پذیرفت، بپذیر؛ در غیر این صورت رد کن.

زمان کل برابر $O(p_1(n) + p_2(n))$ و چندجمله‌ای است. بنابراین $L_1 \cap L_2 \in \mathcal{P}$.

کلاس $\mathcal{NP}$: چون $L_1, L_2 \in \mathcal{NP}$ ، ماشین‌های تورینگ غیرقطعی مانند N_1 و N_2 وجود دارند که در زمان چندجمله‌ای کار می‌کنند. ماشین غیرقطعی N را روی ورودی x به صورت زیر تعریف می‌کنیم:

۱. ماشین $N_1(x)$ را به صورت غیرقطعی اجرا کن. اگر رد کرد، رد کن.

۲. ماشین $N_2(x)$ را به صورت غیرقطعی اجرا کن. اگر پذیرفت، بپذیر؛ در غیر این صورت رد کن.

ورودی x پذیرفته می‌شود اگر و تنها اگر مسیری وجود داشته باشد که هم N_1 و هم N_2 آن را بپذیرند، یعنی $x \in L_1 \cap L_2$. زمان اجرا برابر $O(\text{poly}_1(n) + \text{poly}_2(n))$ است، پس $L_1 \cap L_2 \in \mathcal{NP}$.

ج

کلاس $\mathcal{P}$: برای ورودی x به طول n :

۱. به ازای هر i از 0 تا n :

- رشته x را به دو بخش $y = x[1..i]$ و $z = x[i+1..n]$ تقسیم کن.
- $M_1(y)$ و $M_2(z)$ را اجرا کن.
- اگر هر دو پذیرفتند، بپذیر.

۲. اگر برای هیچ i ای پذیرفته نشد، رد کن.

تعداد تقسیم‌بندی‌ها $n+1$ حالت است و هر تست در زمان چندجمله‌ای انجام می‌شود. زمان کل

$$O(n \cdot (\text{poly}_1(n) + \text{poly}_2(n)))$$

است که چندجمله‌ای می‌باشد. پس $L_1 \cdot L_2 \in \mathcal{P}$.

کلاس $\mathcal{NP}$: ماشین غیرقطعی N روی ورودی x به طول n :

۱. به صورت غیرقطعی یک نقطه شکست $i \in \{0, \dots, n\}$ انتخاب کن تا $x = yz$ شود.
۲. به صورت غیرقطعی $N_1(y)$ و $N_2(z)$ را اجرا کن.
۳. اگر هر دو پذیرفتند، بپذیر.

حدس زدن i و اجرای غیرقطعی هر دو ماشین در زمان چندجمله‌ای انجام می‌شود، پس $L_1 \cdot L_2 \in \mathcal{NP}$.

د

کلاس $\mathcal{P}$: از روش برنامه‌نویسی پویا استفاده می‌کنیم. برای رشته x به طول n : آرایه جدول $T[0..n]$ را تعریف می‌کنیم که $T[i] = \text{True}$ است اگر و تنها اگر پیشوند $x[1..i] \in L^*$ باشد.

۱. مقدار اولیه $T[0] = \text{True}$ قرار بده (چون $\epsilon \in L^*$).

۲. برای $i = 1$ تا n :

- قرار بده $T[i] = \text{False}$.
- برای $j = 0$ تا $i-1$:

– اگر $T[j] = \text{True}$ و $M(x[j+1..i])$ پذیرفته شد، قرار بده $T[i] = \text{True}$ و از حلقه داخلی خارج شو.

۳. مقدار $T[n]$ را خروجی بده.

پیچیدگی زمانی برابر $O(n^2 \cdot \text{poly}(n))$ است که چندجمله‌ای می‌باشد. پس $L^* \in \mathcal{P}$.

کلاس $\mathcal{NP}$: ماشین غیرقطعی N' روی ورودی x به طول n :

۱. اگر $x = \epsilon$ باشد، بپذیر.
۲. به صورت غیرقطعی یک افراز برای x به صورت $x = u_1 u_2 \dots u_k$ حدس بزن ($k \leq n$).
۳. به صورت غیرقطعی $N(u_m)$ را برای تمام $m \in \{1, \dots, k\}$ اجرا کن.
۴. اگر تمامی $N(u_m)$ ها ورودی را پذیرفتند، بپذیر.

حدس زدن افراز و اجرای غیرقطعی روی بخش‌ها در زمان چندجمله‌ای صورت می‌گیرد. پس $L^* \in \mathcal{NP}$.

پاسخ مسئله‌ی ۳.

آ

برای اثبات این‌که مسئله‌ی SUBSET-SUM یک مسئله‌ی NP-complete است، باید دو بخش زیر را نشان دهیم:

۱. عضویت در کلاس NP: ماشین تورینگ غیرقطعی N را روی ورودی $\langle S, t \rangle$ که $S = \{x_1, \dots, x_k\}$ است به صورت زیر تعریف می‌کنیم:

۱. برای هر $i \in \{1, \dots, k\}$ ، به صورت غیرقطعی حدس بزن که آیا x_i در زیرمجموعه انتخاب شود یا خیر.

۲. اعداد انتخاب‌شده را با هم جمع کن و مجموع را با t مقایسه کن.

۳. اگر مجموع برابر t بود، بپذیر؛ در غیر این صورت رد کن.

ورودی پذیرفته می‌شود اگر و تنها اگر زیرمجموعه‌ای با مجموع t وجود داشته باشد. حدس زدن زیرمجموعه و جمع کردن حداکثر k عدد در زمان چندجمله‌ای برحسب طول ورودی انجام می‌شود؛ بنابراین $\text{SUBSET-SUM} \in \text{NP}$.

۲. اثبات NP-Hard بودن (کاهش از 3SAT): مسئله‌ی 3SAT را به SUBSET-SUM کاهش می‌دهیم ($3\text{SAT} \leq_P \text{SUBSET-SUM}$). فرض کنید فرمول نرمال عطف‌دار ϕ دارای n متغیر $x_1, \dots, x_n$ و m بند $C_1, \dots, C_m$ باشد. مجموعه‌ای از اعداد S و هدف t را در مبنای ۱۰ (یا مبنای ۴) با $n + m$ رقم به صورت زیر می‌سازیم:

- برای هر متغیر x_i ، دو عدد v_i و v'_i می‌سازیم. رقم i -ام آن‌ها برابر ۱ است. اگر x_i در بند C_j ظاهر شده باشد، رقم $(n + j)$ -ام عدد v_i برابر ۱ می‌شود، و اگر $\neg x_i$ در بند C_j باشد، رقم $(n + j)$ -ام v'_i برابر ۱ می‌گردد.
- برای هر بند C_j ، دو عدد کمکی s_j و s'_j می‌سازیم که تنها در رقم $(n + j)$ -ام دارای مقدار ۱ هستند.
- عدد هدف t به گونه‌ای تعریف می‌شود که n رقم اول آن برابر ۱ (انتخاب دقیقاً یکی از x_i یا $\neg x_i$) و m رقم بعدی آن برابر ۳ باشند.

این ساختار در زمان چندجمله‌ای برحسب طول فرمول ورودی ساخته می‌شود و ϕ صدق‌پذیر است اگر و تنها اگر زیرمجموعه‌ای وجود داشته باشد که مجموع آن برابر t شود. بنابراین SUBSET-SUM مسئله‌ای NP-hard و در نتیجه NP-complete است.

ب

در نمایش دودویی یا دهدهی، یک عدد طبیعی N با طول $O(\log N)$ بیت نمایش داده می‌شود. در کاهش ارائه شده از 3SAT به SUBSET-SUM، مقادیر اعداد ساخته شده تا مرتبه‌ی 10^{n+m} بزرگ می‌شدند، اما طول نمایش دودویی آن‌ها $O(n+m)$ بود (که چندجمله‌ای است).

اما در نمایش یکانی، عدد N به صورت رشته‌ای از N یک نمایش داده می‌شود؛ یعنی طول ورودی متناظر با خود مقدار عددی N است. اگر همان کاهش را به کار ببریم، نوشتن عدد 10^{n+m} به صورت یکانی نیازمند 10^{n+m} کاراکتر است که نمایی برحسب طول ورودی اولیه $(n+m)$ خواهد بود. بنابراین این کاهش دیگر زمان چندجمله‌ای نخواهد بود و اثبات سخت بودن شکست می‌خورد.

مسئله‌ی مجموع زیرمجموعه‌ها را می‌توان با استفاده از برنامه‌نویسی پویا حل کرد:

- جدول $DP[i][w]$ را طوری تعریف می‌کنیم که نشان دهد آیا با استفاده از زیرمجموعه‌ای از i عدد اول می‌توان به مجموع w رسید یا خیر.

- پیچیدگی زمانی این الگوریتم $O(n \cdot t)$ است که $n = |S|$ تعداد اعداد و t هدف است.

در حالت نمایش یکانی، طول ورودی مسئله برابر است با:

$$L = \text{length}(\text{input}) \geq t + \sum_{i=1}^k x_i$$

زیرا نمایش t به صورت یکانی خود به تنهایی شامل t علامت است. بنابراین $t \leq L$ و $n \leq L$.

در نتیجه، زمان اجرای الگوریتم برنامه‌نویسی پویا برابر است با:

$$O(n \cdot t) \leq O(L \cdot L) = O(L^2)$$

چون زمان اجرای الگوریتم چندجمله‌ای برحسب طول ورودی یکانی است، مسئله در زمان چندجمله‌ای حل می‌شود و بنابراین $\text{UNARY-SSUM} \in \text{P}$.

پاسخ مسئله‌ی ۴.

برای شبیه‌سازی یک ماشین تورینگ T با k نوار توسط ماشین تورینگ M با یک نوار دوطرفه، الفبای نوار ماشین M را به گونه‌ای گسترش می‌دهیم که بتواند موقعیت‌ها و محتویات تمام نوارها را روی یک نوار واحد نگهداری کند.

- به ازای هر علامت X در الفبای نوار T ، یک علامت نشانه‌دار $\dot{X}$ در M در نظر می‌گیریم که نشان می‌دهد نوار مربوطه در آن لحظه روی این خانه قرار دارد.
- محتویات k نوار مختلف بر روی تنها نوار ماشین M با استفاده از یک علامت جداکننده به شکل زیر کنار یکدیگر چیده می‌شوند:

$$\# \dot{a}_{11} a_{12} \dots \# a_{21} \dot{a}_{22} \dots \# \dots \# a_{k1} \dots \dot{a}_{kn} \#$$

برای شبیه‌سازی هر یک گام از محاسبات ماشین T توسط ماشین M :

۱. ماشین M از ابتدای نوار شروع به حرکت کرده و کل نوار را اسکن می‌کند تا موقعیت تمام k هد را پیدا کند و نمادهای زیر آن‌ها را در حافظه حالت خود ذخیره نماید.
۲. پس از خواندن k نماد، ماشین M با استفاده از تابع انتقال ماشین T ، وضعیت بعدی، نمادهای جدید برای جایگزینی، و جهت حرکت هرکدام از هدها را تعیین می‌کند.
۳. ماشین M مجدداً روی نوار حرکت کرده و نمادهای جدید را می‌نویسد، سپس موقعیت هدها را یک خانه به چپ یا راست جابجا می‌کند. اگر هدی به فضای خالی جدیدی نیاز داشته باشد، M محتویات نوار را شیفต์ داده و فضا را باز می‌کند.

فرض کنید ماشین تورینگ T محاسبات خود را در n گام انجام دهد:

- در هر گام از اجرای T ، ماشین M برای پیدا کردن و به‌روزرسانی هدها نیازمند اسکن کردن کل طول نوار فعال خود است.
- طول کل نوار مورد استفاده در M در بدترین حالت متناسب با تعداد کل گام‌های اجراست، یعنی حداکثر در مرتبه $O(n)$.
- بنابراین، برای شبیه‌سازی هر گام از اجرای T ، ماشین M به زمانی معادل $O(n)$ نیاز دارد.
- در نتیجه، کل زمان لازم برای اجرای n گام توسط ماشین تک‌نواره M برابر خواهد بود با:

$$O(n \cdot n) = O(n^2)$$

بنابراین، سربار زمانی روش شبیه‌سازی از مرتبه درجه دوم یا $O(n^2)$ است.

پاسخ مسئله‌ی ۵.

پاسخ مسئله‌ی ۶.